

CONSTRUCTION OF GENE NETWORKS FROM EXPRESSION PROFILES

Ph.D Seminar Report

*Submitted in partial fulfillment of the requirements
for Ph.D Qualifier*

by

Sriram Ponnambalam C

Roll No: 10405602

April 2011

*Department of Computer Science and Engineering
IIT Bombay*

CONTENTS

Contents	i
1 Introduction	1
1.1 Notations	1
2 Gene Regulation and Microarrays	3
2.1 Transcription and Translation	3
2.2 Microarrays	4
2.3 Gene Regulatory Network	5
3 Relevance Networks	7
3.1 Relevance Measures	7
3.2 Structure Learning	8
3.3 Parallel Algorithm	9
3.4 Limitations	10
4 Gaussian Graphical Models	11
4.1 Standard GGM	11
4.2 Improvements	11
4.3 Limitations	12
5 Bayesian Networks	15
5.1 Network Score	16
5.2 Exact Learning	17
5.3 Model Search	18
5.4 Model Averaging	19
6 Module Networks	23
6.1 Probabilistic Agglomerative Clustering	23
6.2 Learning Module Networks	24
7 Performance	27
7.1 Relevance Networks	27
7.2 Gaussian Graphical Models	27
7.3 Bayesian Networks	27
7.4 Module Networks	28
7.5 Comparison Study	28
8 Conclusion	31
Bibliography	33

INTRODUCTION

Biological processes are characterized by complex interactions between thousands of genes and gene products. Identifying these interactions and modeling them is crucial for understanding cellular responses to internal and external signals. Gene expression profiles provide crucial information about these cellular processes. Yet, the regulatory mechanisms of a cell are far from transparent in these data.

Such complex interactions are modeled by gene regulatory networks, construction of which is one of the principal challenges in Systems Biology. Gene Regulatory Network is a mathematical model to explain the observed gene expression levels. Gene regulatory networks are represented as graphs with vertices representing genes and edges representing regulatory interactions between genes. Learning Gene Regulatory Network from multiple expression profiles is an intensely studied research problem.

The problem of reconstructing a gene regulatory network can be formalized as follows: Consider a set of n genes $\{g_1, g_2, \dots, g_n\}$. For each gene, a set of m expression measurements is given. We represent the expression of gene g_i as a random variable X_i with marginal probability $P(X_i)$ derived from some unknown joint probability $P(X_1, \dots, X_n)$. The set $\{X_1, \dots, X_n\}$ is denoted as $\mathcal{X}$. This random variable is described by observations $x_{i,1}, \dots, x_{i,m}$, where $x_{i,j}$ corresponds to the expression level of g_i under condition j . We call the vector $\langle x_{i,1}, x_{i,2}, \dots, x_{i,m} \rangle$ profile of g_i . Given a profile matrix $D_{n \times m}$, $D_{i,j} = x_{i,j}$, we want to find a network G that best explains the data in D . Genes and Variables are used interchangeably in the report.

In this report, we discuss various approaches for constructing a gene network from expression profiles of n genes. In Chapter 2 (p. 3), the required background is covered – the process of gene expression, the methods used to measure gene expression and general properties of gene regulatory networks.

In Chapter 3 (p. 7), we discuss Relevance Networks, methods which construct the network, going over every $n(n-1)/2$ possible edge and use a relevance measure to decide whether to add an edge or not. In Chapter 4 (p. 11), we discuss Gaussian Graphical Models (GGM), which assume that the joint distribution of $\mathcal{X}$ is a multivariate Gaussian distribution, and uses the measure of partial correlation to construct the graph. In Chapter 5 (p. 15), we discuss Bayesian Networks, where the joint distribution $P(\mathcal{X})$ is represented as a directed acyclic graph, and $P(\mathcal{X})$ is decomposed into conditionally independent components as per the Graph. Bayesian Network Learning algorithms construct the structure and the distribution, given data D .

In Chapter 6 (p. 23), we discuss a Module Networks, special case of Bayesian networks, used specifically in constructing gene networks. Module Networks construction assume that genes that are regulated by the same gene probably have the same functionality, and hence have similar expression patterns and conditional probabilities. Finally, in Chapter 7 (p. 27), we discuss the performance studies done for each of the approaches, and also a comparison study done by Grzegorzczuk et al. [2008].

1.1 NOTATIONS

An important notation that is used is that $\tau = 2\pi$, where π is commonly defined as the ratio of circumference of a circle to its diameter. Table 1.1 lists the common notations used in the report.

τ	ratio of circumference of a circle to its radius
n	number of genes/variables
m	number of measurements per gene
g_i	gene i for $i = 1, \dots, n$
D	variable representing the Data matrix dimension $n \times m$
P	probability distribution
$\hat{P}$	estimate of probability distribution
P_X	probability distribution of random variable X - when P is ambiguous
X_i	random variable corresponding to gene i
x_i	a value random variable X_i can take
$P(x_i)$	probability that $P(X_i = x_i)$
$x_{i,k}$	k -th observation of i -th gene/random variable
$\mathcal{X}$	set of random variables $X_1, \dots, X_n$ corresponding to n genes
$E_{P(X)}[v]$	Expectation of v w. r. t. distribution $P(X)$
$1\{r\}$	Indicator function : 1 if r is true, 0 otherwise
(b_{ij})	entry (i,j) of a matrix B
θ, Θ	probability distribution parameters

Table 1.1: Notations

GENE REGULATION AND MICROARRAYS

Gregor Mendel was the first to describe the predictable pattern of inheritance in 1865. By 1903, Mendel's inherited factors (now called genes) were understood to belong to chromosomes, a structure present in each cell of an organism. Even though deoxyribonucleic acid (DNA) was identified as a substance that could possibly be carrying the genetic information as early as 1944, only after the structure of DNA was ultimately elucidated in 1953, it became apparent that DNA is the carrier of genetic information. DNA has two strands, with each strand being a chain or sequence of polymerized nucleotides, called bases – Adenine (A), Guanine(G), Cytosine(C), and Thymine(T). Adenine bonds with Thymine and Guanine with Cytosine making the two strands of DNA complementary. The structure of a gene is defined by the sequence of bases (or base pairs) in its DNA strand. The biological information stored in DNA expresses itself as proteins. In other words, proteins are produced based on the sequence information of DNA. In this chapter we discuss the process by which DNA expresses itself as proteins and, the measurement and regulation of gene expression.



2.1 TRANSCRIPTION AND TRANSLATION

An organism's complete set of genetic information is called its genome. A genome may comprise of several hundred to thousands of genes. Expression of a gene is the process by which the information in the gene's DNA sequence is used to synthesize proteins. A protein is a biological molecule that consists of one or more polypeptides, which are chains of amino acids. Amino acids are one of the simplest biological molecules, and are the building-blocks of proteins. There are 20 different types of amino acids. Since amino acids can be combined in almost any order, and almost in any proportion, proteins are the most structurally variable and therefore the most functionally versatile of the biopolymers.

In gene expression, the molecule called RNA plays an important role. RNA is a single stranded polynucleotide, whose bases are Adenine (A), Guanine(G), Cytosine(C), and Uracil (U). Since it is single stranded, RNA has greater conformational freedom than DNA, because DNA's structure is constrained by the base-pairing (A-T and C-G). While DNA, whose regular structure is suited for long-term storage of genetic information, RNA assumes more active roles in expressing this information.

The idea of a gene itself is defined by its expression of the encoded genetic information. Expression of a gene occurs in two steps : Transcription and Translation. Transcription is a process by which a complementary strand of RNA is derived from the gene. To transcribe a gene, one of the two strands of DNA serves as a template for the synthesis of a complementary strand of RNA. The RNA therefore has the same sequence of DNA – except for the substitution of Uracil (U) for Thymine (T) – and the same orientation. The transcribed RNA is known as messenger RNA (mRNA), because it carries the same genetic message as the gene.

Translation is the process by which a protein molecule is assembled based on the genetic information of mRNA. A cellular particle called the ribosome performs translation of mRNA. At ribosome, small molecules called transfer RNA (tRNA), which carry amino acids, recognize sequential sets of three bases in the mRNA. In other words, a tRNA maps every three consecutive bases to a particular amino acid. The ribosome links the amino acids carried by successive tRNA to form a protein. The

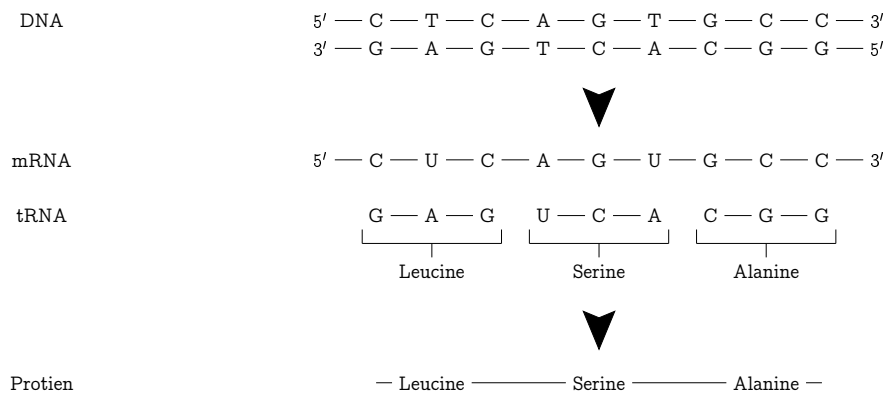


Figure 2.1: Expression of a fragment of a gene is shown. In the transcription step, mRNA is constructed from DNA sequence. In the translation step, with the help of tRNA, protein molecules are constructed from mRNA. Leucine, Serine and Alanine are amino acids corresponding to the sequences GAG, UCA and CGG respectively.

protein's amino acid sequence, therefore, ultimately depends upon the nucleotide sequence of the DNA.

Transcription starts at a site known as the *promoter region* of a gene's DNA sequence. The DNA sequence at this particular site is recognized by specific proteins. Thus, the initiation of transcription requires a set of highly conserved proteins known as transcription factors. Apart from transcription factors, activator and suppressor proteins, which bind in different parts of the DNA, also regulate transcription. For a gene to express itself, it requires that the corresponding regulatory genes are also expressed i.e., expression of a gene can be regulated by other genes.

It has to be noted that regulating transcription is not the only way an organism regulates the expression of genes. Proteins can also be regulated by various post-translation mechanisms, but here we focus only on transcription-based regulation. Reference material for this section is Pratt and Cornely [2004].

2.2 MICROARRAYS

The idea of DNA microarray technology is to monitor gene expression processes by measuring levels of RNA species in biological samples [Polanski and Kimmel, 2007]. RNA levels indicate the expression level of the corresponding gene. The general principle of the procedure is based on hybridization or binding. The idea is to label the RNA by converting back to a single strand of DNA by some reverse transcription mechanism, and then allowing a known complementary DNA sequence of a gene to bind with reverse-transcribed DNA. If the known gene sequence binds with the reverse-transcribed DNA, then the strength of the RNA is the sample proportional to the amount/strength of binding or hybridization. The strength of binding can be measured by dyeing the labeled RNA with fluorescent dyes, and measuring the intensity of fluorescence.

The two main techniques for the measurement of gene expression measurements are (i) complementary-DNA arrays and (ii) oligonucleotide arrays. In both cDNA and oligonucleotide microarrays, labeled (dyed) target RNA or DNA modules bind to fixed complementary DNA probes. In cDNA technology, DNA strands are presynthesized and placed on a glass or plastic plate by micro-robots. This sequence of DNA probes is synthesized based on data from the DNA databases. RNA material from the biological sample is reverse-transcribed back to DNA. The cDNA strands thus obtained are labeled with fluorescent dyes and then presented to DNA array. After the hybridization, the output of a cDNA experiment is the fluorescent intensity on the microarray spots, measured with use of a scanner. In cDNA, the dye intensities are read using image processing techniques. Oligonucleotide microarrays are composed

of DNA strands, that have a fixed length of 20-50 bases, and each gene is represented by 20-25 probes corresponding to its coding fragments. The technology is a combination of solid-phase chemical synthesis and photo-lithography. Oligonucleotide reads are much more reliable than cDNA reads.

The cDNA technology is cheaper and it is possible to implement and develop on laboratory scale. The oligonucleotide technology is more involved and expensive, and there are only a few industrial manufacturers of microarray chips; the most widely known is Affymetrix. In oligonucleotide arrays, the probes are of constant length in the range of 20 – 50 base pairs, while in cDNA arrays the length of cDNA strands differ in spots. In oligonucleotide arrays, the readouts of fluorescence intensities are comparable since the length of complementary DNA is constant, where as in cDNA arrays, the different probe lengths change the reaction between spots. Hence cDNA arrays require the sample to be compared to controls.

2.3 GENE REGULATORY NETWORK

Gene-regulatory network models the complex interactions between genes as a network. Gene regulatory networks are an attempt to model the systems-level organization of complex interactions based on observations from multiple gene expression profiles. Gene regulatory networks are represented as graphs with vertices representing the genes and the edges representing regulatory interactions between genes. Various methods have been developed to learn the gene regulatory network from expression profiles and are discussed in the following chapters.

Assembly of biological interactions into networks reveals that current versions of biological networks are not randomly organized but rather have a *scale-free* format containing hubs with many connections and a large number of nodes that have one or a small number of connections [Zhu et al., 2007]. Random networks have a bell-shaped degree distribution. Scale-free networks, on the other hand, have a typical *power law* distribution, $P(k) \propto k^{-\gamma}$, where k is the degree and $P(k)$ is the probability that a randomly selected node has a degree k . This results in a “fat-tailed” distribution in which there are vertices with high degrees termed “hubs”. The advantage of this type of organization is that the system is more robust; random loss of individual non-hub vertices is less disruptive in a scale-free network than a random network.

Analysis of some transcription factor networks indicate that such networks have a pyramid-shaped hierarchical organization with a few key regulators at the top to which few other factors bind and most transcription factors on the bottom as the functional units for specific pathways.

Analysis of transcription factor-binding data in *E. coli* has revealed three different types of patterns: feed-forward loops (FFL), single input modules (SIM), and dense overlapping regulons (DOR). Among all possible three-element motifs, the FFL was found to be well overrepresented in transcriptional networks.

One of the reasons proposed to explain the development of the scale-free topology is evolution. The probability that a newly added node interacts with an existing node is proportional to its degree, which leads to a so-called preferential attachment model in which rich nodes get richer during evolution and finally form a scale-free network. It has also be noted that some studies indicate that scale-free topology might be generated through experimental design.

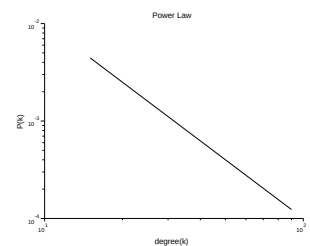


Figure 2.2: Plot of power-law distribution of degree leads to a straight line in log-log scale.

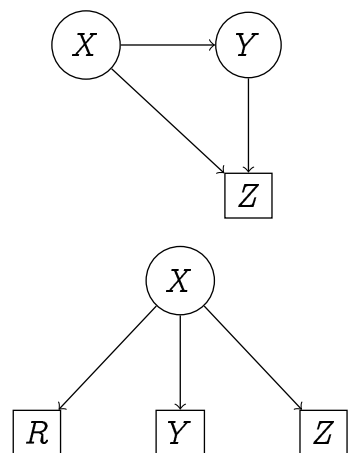


Figure 2.3: Two common motifs of Gene regulatory networks are shown. On the top is Feed Forward Motif, and the bottom is Single Input Module.

RELEVANCE NETWORKS

Relevance Networks are undirected networks, that are constructed based on edge inclusion tests. The idea is to develop a measure of correlation between two genes, and if the measure is significant enough, an edge is added between those two genes. The two main steps here are

1. Define a measure of correlation of two variables X_i and X_j
2. Define a rule to accept or reject edge between X_i and X_j

There are $n(n-1)/2$ possible undirected edges.

One of the earliest measures used in constructing Relevance Network is the *Pearson Correlation Coefficient*. *Mutual Information*(MI) is a better measure than Pearson correlation because it MI measures the degree to which an independence assumption between two variables X_i and X_j is correct. Another score closely related to MI called the χ^2 statistic is also used. The rule of acceptance/rejection is usually based on a threshold value t – any edge $X_i - X_j$ with relevance measure less than t is rejected. Two common methods used to compute the threshold t are Statistical Hypothesis Testing and Permutation testing. We discuss the measurements in § 3.1 (p. 7) and the methods used to compute the threshold in § 3.2 (p. 8). A parallel algorithm developed by Zola et al. [2010] is discussed in the last section.

3.1 RELEVANCE MEASURES

Given two variables X_i and X_j , two different regression lines l_1 and l_2 can be defined: l_1 corresponds to the linear regression of X_j on X_i and l_2 corresponding to the linear regression of X_i on X_j . The angle ϕ , defined between l_1 and l_2 , is an expression of the strength of the linear association between X_i and X_j . The Pearson correlation coefficient ρ_{ij} is the geometrical mean of the slopes of the two regression lines and corresponds to the cosine of ϕ . Higher Pearson correlation between two variables X_i and X_j doesn't necessarily mean a direct interaction between two corresponding genes. It could also mean indirect interaction or regulation by a common gene. Also, ρ_{ij} of 0 doesn't necessarily mean an independence relationship X_i and X_j .

Mutual Information (MI) between two variables X_i and X_j is defined as

$$\mathcal{I}(X_i, X_j) = \mathcal{H}(X_i) + \mathcal{H}(X_j) - \mathcal{H}(X_i, X_j)$$

where differential entropy $\mathcal{H}(X)$ of a continuous variable X is given by

$$\mathcal{H}(X) = - \int P_X(x) \log P_X(x) dx$$

$\mathcal{I}(X_i, X_j)$ is symmetric and non-negative. It can interpreted as a measure of information one variable provides about another. $\mathcal{I}(X_i, X_j) = 0$ if and only if X_i and X_j are independent. The most common methods that are used to estimate Mutual Information are given below:

Gaussian Kernel Estimator is defined by

$$\mathcal{I}(X_i, X_j) = \frac{1}{m} \sum_{X_i, X_j} \log \left(\frac{\hat{P}(X_i, X_j)}{\hat{P}(X_i) \hat{P}(X_j)} \right)$$

3

where the probability distributions are estimated as Gaussian kernels.

$$\hat{P}_{X_i}(x_i) = \frac{1}{\sqrt{\tau}Ns_1} \sum_k e^{-\frac{|x_i - x_{i,k}|^2}{2s_1^2}}; \hat{P}_{X_i, X_j}(x_i, x_j) = \frac{1}{\tau Ns_2^2} \sum_k e^{-\frac{|x_i - x_{i,k}|^2 + |x_j - x_{j,k}|^2}{2s_2^2}}$$

The smoothing parameters s_1 and s_2 are derived from simulations using a wide range of bivariate normal probability densities. Time complexity to compute Gaussian Kernel Estimator is $O(m^2)$.

Histogram Methods estimates the probability density by dividing the observations into fixed number of bins b and then count the number of observations that fall into each bin. Though this technique is fast ($O(m)$), it is imprecise and data sensitive.

B-spline Estimator Daub et al. [2004] proposed to use B-splines as a smoothing criterion for the histogram method. Each observation belongs to k bins simultaneously with weights given by B-spline functions upto order k . It is shown that B-spline estimator produces results correlated with the Gaussian Kernel Estimator. Computational time complexity is $O(m)$.

One of the well known properties of MI is that it is invariant under homeomorphic transformations: $\mathcal{I}(X_i, X_j) = \mathcal{I}(f(X_i), h(X_j))$ for any homeomorphisms f and h . The transformation, where replacing the vector of observations of g_i by the corresponding rank is approximately homeomorphic. Hence, to simplify computing $\mathcal{I}(X_i, X_j)$, $\langle x_{i,1} \dots, x_{i,m} \rangle$ is replaced with $\langle \text{rank}(x_{i,1}), \dots, \text{rank}(x_{i,m}) \rangle$ where $\text{rank}(x_{i,l})$ represents the ranks of $x_{i,l}$ in the set $\{x_{i,1}, \dots, x_{i,m}\}$. Thus, the gene expression vector is a permutation of $\{1, 2, \dots, m\}$, and $\mathcal{I}(X_i, X_j)$ becomes

$$\mathcal{I}(X_i, X_j) = 2\mathcal{H}(\langle 1, \dots, m \rangle) - \mathcal{H}(X_i, X_j)$$

Data Processing Inequality(DPI) is a property of MI that states that if three random variables X_i, X_j, X_k form a Markov Chain in that order, then $\mathcal{I}(X_i, X_k) \leq \mathcal{I}(X_i, X_j)$, which also implies that $\mathcal{I}(X_i, X_k) \leq \mathcal{I}(X_j, X_k)$. DPI is used to eliminate edges that are added due to indirect relationship : each time of pair (X_i, X_k) satisfies both inequalities for any j and the corresponding edge $g_i - g_k$ is removed.

Other measurements such as conditional mutual information i.e., $\mathcal{I}(X_i|Y, X_j|Y)$ are also used. $\mathcal{I}(X_i|Y, X_j|Y)$ verifies conditional independence of two variables X_i and X_j given a set of variables Y .

3.2 STRUCTURE LEARNING

Butte and Kohane [2000] were the first to use Mutual Information as a relevance measure. They used Histogram method to compute $\mathcal{I}(X_i, X_j)$. They defined a threshold t , and an edge is between X_i and X_j when $\mathcal{I}(X_i, X_j)$ exceeds t . They experimented with various values of t and understood the behavior of the network. Later, methods of statistical significance were used to determine the threshold t . We discuss the two most common methods used to compute statistically significant threshold in this section. Here, we denote the relevance measure between two random variables by $d(X_i, X_j)$ and t as the threshold value used for a deciding the significance of a given relevance measure.

3.2.1 Hypothesis Testing

Reference for this sub-section is from Koller and Friedman [2009]. In Hypothesis testing, a null hypothesis H_0 is declared that “data were sampled from a distribution where $P(X_i, X_j) = P(X_i)P(X_j)$ ”. We need to define a decision rule $R(D, t)$ that will accept the input data D , and return verdict either “Accept”, in case H_0 is true or “Reject” otherwise. A common approach is to analyze the probability of false rejection p . Note

that we can not evaluate the probability of false acceptance, because that would imply an assumption on data sets that violate H_0 . The decision rule $R(D, t)$ is of the form

$$R(D, t) = \begin{cases} \text{Accept} & d(D) \leq t \\ \text{Reject} & d(D) > t \end{cases}$$

p -value of a threshold t is defined as the false rejection probability when t is selected for the decision rule $R(D, t)$. Typical significance level used is 95% i.e. null hypothesis is rejected if the relevance measure in the observed data has p -value 0.05 or less. When we reject the hypothesis for X_i and X_j , we add an edge $X_i - X_j$ in the graph.

In an *exact test*, we consider a distribution $P(D|H_0)$, which is the probability of seeing a dataset D given H_0 . Since we care only about the sufficient statistics of X_i and X_j (as relevance measure estimators are generally defined in terms of sufficient statistics¹), we consider the distribution $P(S[X_i, X_j]|H_0, m)$, where $S[X_i, X_j]$ is the sufficient statistics of m samples of X_i and X_j . If we consider $\mathcal{S}_{X_i, X_j}^m$ as the space of all empirical counts over X_i and X_j that might be observed in a dataset of m samples, then

$$p\text{-value}(t) = \sum_{S[X_i, X_j] \in \mathcal{S}} 1\{d(X_i, X_j) > t\} P(S[X_i, X_j]|H_0, m)$$

$1\{\cdot\}$ is the indicator function. $P(S[X_i, X_j]|H_0, m)$ can be computed for small values of m . A more common approach is to use the asymptotic distribution of the sample sufficient statistics under null hypothesis. χ^2 distribution is commonly used as the asymptotic distribution, and the tail probability of this distribution can be used to approximate the p -value for null hypothesis tests.

Basso et al. [2005] developed a method called *ARACANE*, which used Mutual Information with Gaussian Kernel Estimators and applied Hypothesis testing for construction of regulatory networks in human B cells. *ARACANE* also used DPI to eliminated edges from indirect relationship. *ARACANE* successfully constructed a gene network that shows a power-law tail in the relationship between the number of genes n and their number of interactions. Though they claim that it offers much better results, in terms of sensitivity, compared to Bayesian networks, Hartemink [2005] showed an instance where Bayesian networks are, in fact, better.

3.2.2 Permutation Testing

Let $\pi(X_i) = \pi(x_{i,1}, \dots, x_{i,m})$ denote a permutation of the vector of m observations. Permutation testing method [Zola et al., 2010] involves computing $\mathcal{I}(\pi(X_i), X_j)$ for all possible $m!$ permutations of X_i and accept the dependency between X_i and X_j only if $\mathcal{I}(X_i, X_j) > \mathcal{I}(\pi(X_i), X_j)$ for a at least $1 - \epsilon$ of $m!$ permutations. Here ϵ depends upon the required statistical significance (computed using the tail of asymptotic distributions). Since testing all $m!$ permutations is computationally expensive, a large sampling of permutation space works well in practice. One can view permutation testing as ‘‘Hypothesis Testing by sampling’’. Also, Zola et al. [2010] identified that the rank transformation makes each permutation test valid for all the $n(n-1)/2$ pairs of observations, thus reducing the number of MI computations for permutation testing by $O(n^2)$.

3.3 PARALLEL ALGORITHM

Zola et al. [2010] developed a parallel algorithm that uses the B-spline method to compute MI and applies Permutation Testing for estimating edge-inclusion test. Let p be the number of processors. The graph denoted by $n \times n$ adjacency matrix A . The algorithm use row-wise data distribution where each processor stores up row-wise distribution – each processor stores up to $\lceil \frac{n}{p} \rceil$ rows of the matrix D . Entry (i, j) in A has X_i as its row gene and X_j as its column gene. After data D is parsed and each processor receives its block of data, the algorithm progresses in three phases

1. In statistics, a sufficient statistic is a statistic which has the property of sufficiency w.r.t. a statistical model. It has the interpretation that ‘‘no other statistic which can be calculated from the same sample provides any additional information as to the value of the model’s parameter’’. For example, for a Gaussian distribution model with unknown mean and variance, sample mean and sample variance are sufficient statistics.

p_0	0	1	2		
p_1		0	1	2	
p_2			0	1	2
p_3	2			0	1
p_4	1	2			0

Figure 3.1: Block partition scheme for 5 processors. For each block, the iteration number is shown.

1. In the first phase, MI is computed for all the possible pairs of genes, and q randomly chosen permutations per pair. Each processor is assigned a sub-matrix $A_{i,j}$ ($0 \leq i, j < p$) of size $\frac{n}{p} \times \frac{n}{p}$ each. The processor computes MI values in its sub-matrix and q random permutations for each position. To compute it, each processor requires the expression profile vectors of all genes representing rows or columns in the sub-matrix. Note that the task assignment is done by block decomposition of A , the data assignment is done by row-wise decomposition of D .

For blocks in the main diagonal of A , the row and column genes are the same, while for other blocks, they are distinct. Since A is symmetric, it is sufficient to compute only one-half of A . The assignment of submatrices to processors is as follows: In iteration i , processor with rank j , $0 \leq j < p$ is assigned the sub-matrix $A_{j,j+i \bmod p}$ (Figure 3.1). In iteration 0, MI values for diagonal blocks of A are computed. No communication is necessary for iteration 0 since each processor has the row and column profile vectors. Then, at end of the iteration column gene vectors are shifted up i.e., processor with rank j sends its column gene vectors to processor with rank $(j - 1 + p) \bmod p$, after which the next set of blocks are computed. Iteration continues until one-half of the adjacency matrix A is covered. It takes compute time $O(qn^2m/p)$ and communication time $O(\mu nm + p\nu)$, where ν is the latency and bandwidth is $\frac{1}{\mu}$.

2. To compute the threshold, we need to find an element with rank $(1 - \epsilon) \cdot q \cdot n(n - 1)/2$ of the previously computed values. Since ϵ is close to zero, the threshold value is close to the largest value in the sorted order of the computed MI values. Each processor sorts all the values it computes and picks the $r = \epsilon \cdot q \cdot n(n - 1)/2$ largest values. A parallel reduction is applied using the r largest values in each processor, and r -th largest value of all the MI values is computed. Once r -th largest value, t , is computed, each processor eliminates edges from its local adjacency matrix, if the edge's corresponding MI is less than t . This phase takes $O((qn^2 \log n + r \log n)/p)$ compute time, and $O((\nu + \mu r) \log p)$ communication time.
3. In this third phase, DPI is applied to eliminate indirect edges. To compute DPI for $D_{i,j}$, complete information about rows i and j are needed. Since matrix is stored row-wise, all information about row j should be streamed to the processor responsible for row i . The computation is done in $p - 1$ rounds, where in round i , only processors with ranks $0, 1, \dots, p - i$ participate in communication and processing. Even though the parallel run-time of this phase is $O(\frac{n^3}{p})$, this phase takes the least time, since the graph is sparse.

3.4 LIMITATIONS

A few of the limitations of Relevance networks are [Basso et al., 2005]

- Network show no direction, so it is not known whether X_i is dependent on X_j or otherwise.
- DPI incorrectly removes the direct edges in some cases.
- Some times correlation between two genes may involves some unknown intermediates, whose expression values haven't been observed. Relevance networks can not handle hidden variables.
- Since we are using DPI to eliminate indirect edges, three-gene loops (for example, feedback mechanisms) can not be expressed in Relevance networks.
- When the expression levels of a gene is very low, the method can not produce reliable results.

GAUSSIAN GRAPHICAL MODELS

4

Gaussian Graphical Models (GGM) represent probability relationships of a multivariate Gaussian distribution using an undirected graph. Under this approach, the observed data matrix D of dimension $n \times m$, with m samples of n genes each, is assumed to be drawn from a multivariate normal distribution $\mathcal{N}(\mu, \Sigma)$ with some mean vector $\mu = (\mu_1, \dots, \mu_n)^T$ and positive definite co-variance matrix $\Sigma = (\sigma_{ij})$. The standard algorithm to construct GGM from data is based on the assumption that $m \gg n$. But in the case of gene expression data, $n > m$. In this section, we discuss the improvements to GGM made by Schäfer and Strimmer [2005] for constructing gene networks. Note that in this chapter the notation π is used for partial correlation coefficient.

4.1 STANDARD GGM

In GGMs, the strength of pairwise correlation is characterized by the partial correlation matrix $\Pi = (\pi_{ij})$, which is the correlation between two variables X_i and X_j conditioned on all remainder of the variables. For example, the partial correlation π_{12} between variables X_1 and X_2 is the correlation $\text{corr}(\epsilon_1, \epsilon_2)$ of the residuals ϵ_1 and ϵ_2 resulting from linearly regressing variables X_1 and X_2 on $X_3, \dots, X_n$ respectively. Note that π_{ij} is the correlation coefficient of the conditional bi-variate distribution of variables X_i and X_j i.e., $P(X_i, X_j | \mathcal{X} - \{X_i, X_j\})$. It can be shown that two variables X_i and X_j of a multivariate Gaussian $\mathcal{X}$ are conditionally independent given $\mathcal{X} - \{X_i, X_j\}$ if and only if π_{ij} vanishes.

Using the definition $\sigma_{ij} = \rho_{ij}\sigma_i\sigma_j$, the covariance matrix Σ can be further decomposed into variance components σ_i^2 and the Pearson correlation coefficient matrix $B = (\rho_{ij})$. Pearson correlation is discussed in the section § 3.1 (p. 7). By standard graphical model theory, $\Omega = B^{-1} = (\omega_{ij})$ and $\pi_{ij} = -\omega_{ij} / \sqrt{\omega_{ii}\omega_{jj}}$. It is also valid to use Σ instead of B here.

The standard algorithm to build Gaussian Graphical Models, which assumes $m \gg n$, is as follows:

- 1 Estimate correlation matrix from unbiased sample covariance matrix : $\hat{\Sigma} = (\hat{\sigma}_{ij}) = \frac{1}{m-1} \sum_{i=1}^m (D_{i\cdot} - \hat{\mu})(D_{i\cdot} - \hat{\mu})^T$, where $D_{i\cdot}$ is the i -th row of D , and $\hat{\mu}$ is the sample mean vector.
- 2 Compute partial correlation coefficients using equations given above.
- 3 Use Hypothesis testing to compute which correlations are significantly different from zero. (similar to § 3.2 (p. 8)).
- 4 Visualize the inferred correlation structure by a graph.

However the standard algorithm is not applicable for gene networks, because $n > m$ in gene expression data.

4.2 IMPROVEMENTS

Schäfer and Strimmer [2005] improved the standard algorithm in three ways. First in order to compute correlation, they used the Moore-Penrose pseudo-inverse and Bagging. Second, they developed a mixture distribution for computing the statistical

significance of a partial correlation. Third, they employed false discovery rate multiple testing to avoid exhaustive search among all the graphical models.

Moore-Penrose pseudo-inverse is a generalization of the standard matrix inverse, so that, it can be applied to singular matrices and is based on singular value decomposition (SVD). The correlation matrix B can be decomposed into $B = U\mathcal{D}V^T$, where $\mathcal{D}$ is a square diagonal matrix of rank $r \leq \min(n, m)$ having all non-zero singular values. The pseudo-inverse B^+ is defined as $B^+ = V\mathcal{D}^{-1}U^T$. Schäfer and Strimmer [2005] used B^+ to estimate Π . In order improve upon the unstable estimate (since $n \gg m$), they used the idea of Bagging as below:

- 1 Generate a bootstrap sample D^b with replacement from the original data
- 2 For each data sample calculate the estimate $\hat{\theta}^b$
- 3 Repeat above steps for B times and the bagged estimate is given by $(1/B) \sum_b \hat{\theta}^b$

Schäfer and Strimmer [2005] developed three different estimates of Π by different combinations of bagging and pseudo-inverse. They find that accuracy varies for each estimate.

The standard method of estimating statistical significance of partial correlation is to use the principle of Hypothesis Testing (as discussed in § 3.2 (p. 8)). Here H_0 defined as $\pi_{ij} = 0$. Distribution of sample normal correlation coefficient is well known in Statistics. When $\rho = 0$ in the actual distribution, the distribution of sample estimates is derived as

$$P_0(\hat{\rho}|k) = (1 - \hat{\rho}^2)^{(k-3)/2} \sqrt{\frac{2}{\pi}} \frac{\Gamma(k/2)}{\Gamma((k-1)/2)}$$

Partial correlation coefficient is distributed similar to correlation coefficient with $k = m - n + 1$, but it can not be used in small-sample testing, since it assumes $m > n$. Schäfer and Strimmer [2005] utilized the fact that bio-molecular networks are typically sparse and developed a mixture distribution for the distribution of π . Since only a fraction η_A of $n(n-1)/2$ pairs will correspond to actual edges, the following mixture distribution of partial correlation coefficients π is assumed across all edges in the network.

$$P(\pi) = \eta_0 P_0(\pi|k) + \eta_A P_A(\pi)$$

where P_0 and P_A are the null and alternative distributions respectively and $\eta_0 + \eta_A = 1$. P_A is assumed to be a simple uniform distribution from -1 to 1 . The parameters η_0 and k are learned by fitting the observed partial correlation coefficients, from which p -values for each possible $n(n-1)/2$ edge can be calculated using the exact null distribution P_0 . Alternatively one may also compute

$$P(\text{non-zero edge} | \pi) = \frac{\hat{\eta}_A P_A(\pi)}{P_0(\pi|k)}$$

Finally, Schäfer and Strimmer [2005] use false discovery rate(FDR) multiple testing to test each of the potential $n(n-1)/2$ edges for their presence in the final network. The basic algorithm is as follows

- 1 Construct the set of ordered p -values $p_1, \dots, p_E$ corresponding to edges $e_1, \dots, e_E$
- 2 Let i_Q be the largest value for which $p_i \leq (i/E)(Q/\eta_0)$.
- 3 Reject the null hypothesis of zero partial correlation for all edges $e_1, \dots, e_{i_Q}$.

It has been shown that the above procedure controls FDR at level Q i.e. the expected proportion of false positives out of the total number of rejections is controlled at level Q

4.3 LIMITATIONS

The limitations identified with GGMs are

- Most critical argument against GGM is the assumption of linear relationships among the investigated variables.
- It is possible that there may be higher order interactions among more than two variables. GGMs evaluate only pairwise interactions only.

Schäfer and Strimmer [2005] also note that Bayesian networks are better than GGMs as the former can model non-linear relationships. Their argument for GGM is that

...to model gene association and dependence on small-sample data sets it is prudent to choose a graphical model (such as a GGM) that requires very few assumptions and only a minimal number of parameters.

BAYESIAN NETWORKS

Bayesian networks are graphical models of probability that represent relationships among a set of dependent random variables. Bayesian Networks are used in various applications such as image processing, spam filtering, bioinformatics, recommendation engines etc. In Bayesian networks, relationships between random variables are shown in terms of conditionally independent components of the joint distribution of all the variables. Formally a Bayesian network of a set of variables $\mathcal{X} = \{X_1, \dots, X_n\}$ is a pair (G, P) , where G is a directed acyclic graph $(\mathcal{X}, E)$ and P is the joint probability distribution of $\mathcal{X}$. In G , a node X_i is a *parent* of X_j if $(X_i, X_j) \in E$. If there is a path from X_i to X_k , then X_k is a *descendant* of X_i and X_i is an *ancestor* of X_k . In a Bayesian network, the following assumption holds: For each variable X_i , given its parents $Pa(X_i)$, X_i is independent of all its non-descendants. The joint distribution $P(\mathcal{X})$ can be written as

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i | Pa(X_i), \theta_i)$$

where θ_i is set of parameters for the conditional distribution $P(X_i | Pa(X_i))$. We denote Θ_G to represent the set of parameters θ_i of all conditional distributions of G .

To learn a Bayesian network G from data D , we need to identify both the structure of the network and the parameters of each conditional distribution $P(X_i | Pa(X_i))$ defined by the network. Suppose the structure of a network is known, then the problem is to learn the parameters. In a Bayesian approach of parameter learning, we assume a prior distribution $P(\theta_i)$ for the parameters of each $P(X_i | Pa(X_i))$. For example, if $P(X_i | Pa(X_i))$ assumed to be a Multinomial distribution, then Dirichlet distribution is assumed to be the prior for Multinomial's parameters themselves. A Bayesian approach for estimating the parameters for $P(X_i | Pa(X_i))$ is to first estimate the posterior distribution of parameters $P(\theta_i | D)$ and assess θ_i by averaging the posterior for all possible θ_i i.e., by computing $E_{P(\theta_i | D)}[\theta_i]$ (Figure 5.2). By the chain rule of probability, the posterior $P(\theta_i | D) \propto P(D | \theta_i)P(\theta_i)$, where $P(D | \theta_i)$ is the likelihood term. If the conditional distributions are of certain exponential family of distributions (which includes both Multinomial and Gaussian) and if the parameter distributions are independent, then the parameters of conditional distributions of a Bayesian Network can be estimated in closed form (assuming an appropriate prior).

In order to discover the structure of the network, we need a score to compare two networks on how better a network can explain data. A Bayesian approach on structure discovery is as follows. Let $P(G)$ be the distribution of networks. We are interested in $P(G | D)$. Applying the chain rule of probability, we have $P(G | D) \propto P(D | G)P(G)$. Score of a Bayesian network is defined as $\log P(D | G) + \log P(G)$. Here $P(D | G)$ is computed using a Bayesian approach similar to parameter learning, but difference here is that the likelihood $P(D | \Theta_G, G)$ is averaged over all possible Θ_G i.e., by computing $E_{P(\Theta_G | G)}[P(D | \Theta_G, G)]$. $P(G)$ is usually taken to be uniform. It turns out that with certain assumptions, the score can be decomposed into sum of local scores – with each local score corresponding to $P(X_i | Pa(X_i))$. Further discussion about the score is given in the § 5.1 (p. 16).

Having defined the score for a given network, the task is to find a network with the best score. Unfortunately, in the general case, this problem turns out to be NP-Hard, even when the number of parents [Chickering, 2004] are bounded. Many methods

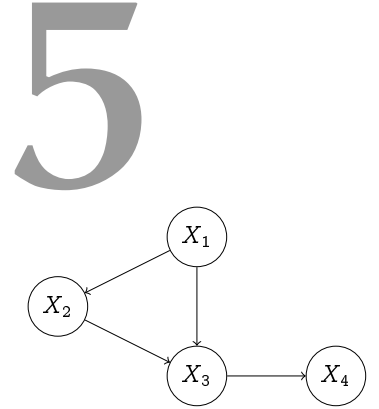


Figure 5.1: A Bayesian Network with 4 variables $P(\mathcal{X}) = P(X_1)P(X_2|X_1)P(X_3|X_1, X_2)P(X_4|X_3)$

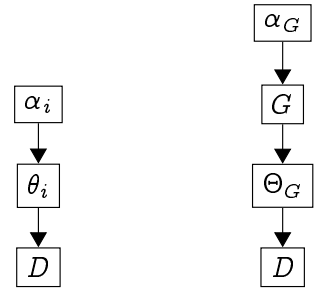


Figure 5.2: Model Hierarchy of the Bayes Network. On the left, model hierarchy for $P(X_i | Pa(X_i))$ is shown. $P(\theta_i | D) \propto P(D | \theta_i)P(\theta_i)$. On the right, model hierarchy for $P(G | D)$ is shown. $P(G | D) \propto P(D | \Theta_G, G)P(\Theta_G | G)P(G)$. α is the prior in both cases.

have been developed to exploit the properties of either the Graph or the nature of data in order to learn the structure in practical cases. Exact learning of Bayesian networks exploits the topological sort property of directed acyclic graph. Heuristic methods to search the graph space such as Greedy Structure search exploit the nature of data and the decomposable property of the score. Model averaging methods attempt to develop confidence estimate of certain features of a network, such as the existence of edge between two variables, by averaging over a sample from the distribution $P(G|D)$. We will discuss these methods in this chapter.

5.1 NETWORK SCORE

The purpose of a network scoring metric is to compare two networks such that a network that explains the data better has a better score. Scoring Metric for Bayesian Network is defined as

$$\text{score}(G, D) = \log P(D|G) + \log P(G)$$

The term $P(D|G)$ is called marginal likelihood, and $P(G)$ is the prior. In a true Bayesian approach, it would be computed as $E_{P(\Theta_G|G)}[P(D|\Theta_G, G)]$. To compute the score efficiently, the following set of assumptions are usually made [Heckerman, 1999].

Parameter Independence assumption states that for network structures G

$$P(\Theta_G|G, D) = \prod_{i=1}^n P(\theta_i|G, D)$$

With the assumption of *Parameter Independence*, we can update each set of parameters θ_i independently. For example, if we are interested in estimating the parameter for each distribution then $E_{P(\Theta_G|G, D)}[\Theta_G] = \prod_{i=1}^n E_{P(\theta_i|G, D)}[\theta_i]$.

Parameter Modularity assumption states that if a variable X_i has the same set of parents in two networks G_1 and G_2 , then the parameters priors are the same.

$$P(\theta_i|G_1, D) = P(\theta_i|G_2, D) \text{ if } Pa(X_i) \text{ is the same in } G_1 \text{ and } G_2$$

Structure Modularity assumption states that the prior $P(G)$ is defined as a product of terms, with each term corresponding to $Pa(X_i)$.

$$P(G) \propto \prod_i P(Pa(X_i))$$

Dirichlet distribution is commonly used as a prior for Multinomial distribution. Dirichlet distribution of K parameters is given by

$$P(\theta_i|\alpha_i) = \frac{1}{B(\alpha_i)} \prod_{j=1}^K \theta_{ij}^{\alpha_{ij}-1}$$

where $B(\alpha_i)$ is a normalizing constant. Here α_i s are the hyper-parameters (parameters of the parameter priors). In the context to the Bayesian approach, they are also called *equivalent sample sizes*. In the case of Dirichlet priors, $P(D|G)$ this can be computed in a closed form [Heckerman and Chickering, 1993]. Combined with the assumptions of Structure modularity and Parameter modularity, it can be shown that

$$\text{score}(G, D) = \sum_i \text{LocalScore}(X_i, Pa(X_i), D)$$

where $\text{LocalScore}(X, U, D)$ measure how well a set of variables U set of parents of X for dataset D .

If we were to specify hyper-parameters for each $X_i, Pa(X_i)$ combination, then number of hyper-parameters is super-exponential. To avoid this we define a prior distribution over the entire probability space $P'(X_1, \dots, X_n)$ and a sample size α for the set of imaginary samples. Now for each prior $P(X_i|Pa(X_i))$, hyper-parameters are computed as

$$\alpha_{X_i|Pa(X_i)} = \alpha \cdot P'(X_i, Pa(X_i))$$

P' can be represented by another Bayesian network, but is commonly defined as a set of independent marginals. The score computed by such a representation of priors is called the score of “Bayesian networks with Dirichlet and equivalence assumption”, in short, *BDe*. A similar score is also developed for Gaussian networks called *BGe* score by Geiger and Heckerman [1994].

Bayesian Information Criterion, an approximation of the marginal likelihood, is commonly used as a network score. It assumes that all parameters in the network have a Dirichlet prior and computes the score as

$$\log P(D|G) \approx \log P(D|\Theta_G, G) - \frac{d}{2} \log m$$

where d is the number of independent parameters in G . Here $\log P(D|\Theta_G, G)$ is the log likelihood of the data given Θ_G and G , and Θ_G is computed from Bayesian approach (as $E_{P(\Theta_G|G,D)}[\Theta_G]$) as explained above. BIC is an approximation of the actual marginal likelihood, and it can be proved that as $m \rightarrow \infty$, $\log P(D|G) \rightarrow \text{BIC}(D, G)$. The negation of this quantity can be viewed as the number of bits required to encode both the model and the data given the model. Hence, this score is also known as *Minimum Description Length*

5.2 EXACT LEARNING

Search space for a gene network of n vertices is the space of directed acyclic graphs with n vertices. The number of directed acyclic graphs is derived as $O(n!2^{O(n^2)})$. Exact Learning method finds the optimal solution in this super-exponential space in exponential time. Here, we discuss the algorithm developed by Ott et al. [2004].

Let S_G be the set of genes $\{g_1, \dots, g_n\}$ and a network score function as described earlier is available i.e., LocalScore function that takes input a variable g and a set of parents $Pa(g)$, and assigns a score. The score of the network is computed as the sum of local scores of every gene. The task of learning the network is to find a set of parent genes for each gene, such that the acyclic network of the genes has the maximum score i.e., $\max_G \text{score}(G)$. Let $F(X_i, A)$ denote the highest possible score that can be obtained by choosing parents of X_i from A i.e., $\max_{B \subseteq A} \text{LocalScore}(X_i, B, D)$. A recursive formulation for F is

$$F(X_i, A) = \max \begin{cases} \text{LocalScore}(X_i, A, D) \\ \max_{X_j \in A} F(X_i, A - \{X_j\}) \end{cases}$$

Every DAG has at least one topological sorted order of its nodes or an ordering of nodes has at least one DAG consistent with the order. The correspondence between a DAG and order is used to build a recursive formulation for $\max_G \text{score}(G)$. An ordering of A is defined by $\pi(A)$, a permutation of elements in the set A . Given a $A \subseteq \mathcal{X}$, and an order $\pi(A)$, Let $Q(A, \pi(A))$ be the optimal score of network on A consistent with $\pi(A)$ computed as

$$Q(A, \pi(A)) = \sum_{X_i \in A} F(X_i, \{X_j | X_j \text{ precedes } X_i \text{ in } \pi(A)\})$$

The goal of the algorithm is to find $M(\mathcal{X})$, where $M(A)$ is defined as

$$M(A) = \arg \max_{\pi' \in \pi(A)} Q(A, \pi')$$

Algorithm is as follows

```

for  $A \subseteq S_G$  ordered by increasing size of  $A$ 
do  $\forall X_i \in A$ 
     $F(X_i, A) \leftarrow \min\{\text{LocalScore}(X_i, A, D), \min_{X_j \in A} F(X_i, A - \{X_j\})\}$ 
for  $A \subseteq S_G$  ordered by increasing size of  $A$ 
do
     $g'(A) \leftarrow \arg \min_{X_i \in A} F(X_i, A - \{X_i\}) + Q(A - \{X_i\}, M(A - \{X_i\}))$ 
     $M(A)[i] \leftarrow M(A - \{g'(A)\})(i)$  for  $1 \leq i < |A|$ 
     $M(A)[|A|] \leftarrow g'(A)$ 

```

Time Complexity of this algorithm is $O(n \cdot 2^n)$.

Nikolova et al. [2009] developed a parallel algorithm based on Ott et al. [2004]. We can visualize the dynamic programming algorithm as operating on the lattice L formed by the partial order “set inclusion” on the power set of $\mathcal{X}$. Lattice is a directed graph (V, E) , where $V = 2^{\mathcal{X}}$ and $(B, A) \in E$ if $B \subset A$ and $|A| = |B| + 1$. Lattice is naturally partitioned into levels $l \in [0, n]$ with each level l containing all the subsets of size l .

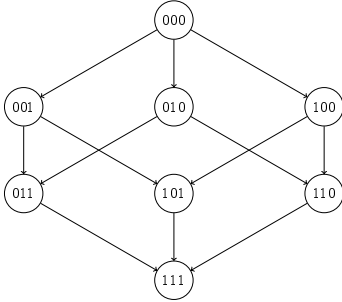


Figure 5.3: Lattice for 3 variables. Subset of each node is shown with bit string representation. Arrows show the direction of data transfer from each node.

Each node in L , representing a subset A at level $|A| = l$, has l incoming edges from nodes $A - \{X_i\}$ for each $X_i \in A$ and $n - l$ outgoing edges to $A \cup X_i$ for each $X_i \notin A$. When mapping the algorithm to lattice, we can see that $n - l$ F functions, $g'(A)$ and $M(A)$ are computed at each node of the lattice. Each level in the lattice can be computed concurrently, as the computation at level l depends only on value computed at level $l - 1$.

We can note that L is an n -dimensional hypercube, and thus can map the parallelization to an n -dimensional hypercube. The n -D hypercube algorithm runs in $n + 1$ steps. Let ω_A denote the id of a processor. The bit string corresponding to binary representation of ω_A corresponds to a subset A of $\mathcal{X}$. Let $\mu(\omega_A)$ denote the number of 1's in ω_A . $\mu(\omega_A)$ correspond to the level l of the subset A . Each processor is active in only one time step, and processor ω_A is active in time step $\mu(\omega_A)$. It receives $(n\mu(\omega_A) + 1)$ F values and one $g'(A)$ value from each of its $\mu(\omega_A)$ neighbor. Run time of each step is $O(n - i)$. Using 2^n processors, we can compute in $\sum O((n - i)i) = O(n^3)$.

Let $p = 2^k$ be the number of processors available. Nikolova et al. [2009]'s strategy is to decompose the n -D hypercube into 2^{n-k} k -D hypercubes and map each k -D hypercube to the $p = 2^k$ processors. With such a mapping and pipelining of k -D hypercubes, the algorithm takes $2^{n-k} + k$ time steps such that all processors are active except for the first k and last k time steps during the build up and finishing off of the pipeline. For correctness, pipelining requires that computation of node A in the hypercube H_i starts before computation of node B in the hypercube H_j if and only if $\mu(\omega_A[1, nk]) < \mu(\omega_B[1, nk])$ (or) $\mu(\omega_A[1, nk]) = \mu(\omega_B[1, nk])$ and ω_A is lexicographically smaller than ω_B . Here $\omega_A[i, j]$ is the sub-string from position i to j of the bit representation of ω_A . Parallel run-time of the algorithm is $O(n^2 \cdot 2^{n-k})$.

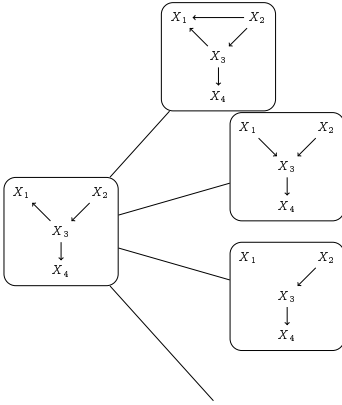


Figure 5.4: Graph neighborhood for a graph with 4 variables. An example of addition, reversal and removal is shown.

1. Neighbors are all of the legal networks obtained by a single operator to G

5.3 MODEL SEARCH

In general case, given a data set D and decomposable score function the problem of finding a graph G that maximizes the score is NP-Hard. Hence, heuristic algorithms, that attempt to find the best network by exploring the search space by local search, are used. But such methods don't guarantee to find the best solution. Search space can be viewed as a graph over candidate solutions, connected by possible operators that a search procedure can perform to move between different solutions.

The most common operators used in model search, that move from one candidate to another are: edge addition, edge deletion and edge reversal. The advantage of the above operators are one, the search space of an operator is at most n^2 (i.e., each operator has at most n^2 possible moves) and two, the operations will result in change only one local term (since the score of network is a sum of local scores).

In the structure learning setting, the search procedure picks up an initial network structure G as a starting point. It then considers all its neighbors¹ and compute the

score for each of them. Then, it applies the changes that lead to best improvement in score. It repeats this process until no modifications improve the score.

If K is the number of iterations for convergence, this procedure takes $O(Kn^2)$ operator applications. Each step involves computing the sufficient statistics for the operator application, which depends upon the operator – upper bounded by the size of data. When the local search operator space of $O(n^2)$ during each iteration becomes too expensive, instead of exhaustive enumeration, a randomized choice of operators is used. Another problem with the search procedure is that the search procedure might reach and stay at a local maximum. The common heuristics used to avoid local maximum are random restarts, where the procedure is repeated starting at different initial graphs at random, and tabu search, where a list of recently applied operators are maintained to avoid applying the corresponding reverse operators again.

5.4 MODEL AVERAGING

In Bayesian Networks, several networks have similar scores. This suggests that instead of picking one of structure and drawing conclusions about the domain, one should develop *confidence estimates* about the conclusions made by a discovered structure. Hence, in the methods of model averaging, one attempts to learn *features* that are common to the networks describing the data.

The most common features of interest in a Bayesian Network are

Markov Feature Is X_j in the Markov blanket of X_i ? The Markov blanket of X_i is the minimal set of variables that shield X_i from the rest of the remaining variables – denoted by $MB(X_i)$. $X_j \in MB(X_i)$ if G contains the edge $X_j \rightarrow X_i$ or the edge $X_i \rightarrow X_j$ or if there is a variable X_k such that both the edges $X_i \rightarrow X_k$ and $X_j \rightarrow X_k$ are in G .

Order Relations Is X_i an ancestor of X_j ? This is a global property, and it is an indication, not necessarily an evidence, that X_i might be a causal ancestor of X_j .

Edge Feature What is the posterior probability of U being the parent set of X_i ?

Parent Set Feature Does X_j belong to $Pa(X_i)$?

All of the features are to be estimated in terms of probability.

Given a feature $f(G)$ of a graph, we are interested in computing

$$E_{P(G|D)}[f(G)] = \sum_G f(G)P(G|D)$$

Bayesian score, defined in the section § 5.1 (p. 16), is proportional to $\log P(G|D)$. Hence

$$P(G|D) \propto \prod_i \text{LocalScore}(X_i, Pa(X_i), D)$$

A model averaging method samples from the distribution $P(G|D)$ and computes an estimate for $E_{P(G|D)}[f(G)]$ from the samples. It is an Monte-Carlo method on the distribution $P(G|D)$.

5.4.1 Structure MCMC

In general, an MCMC sampling scheme can be used to generate sample $s_1, s_2, \dots$ from a discrete target distribution $P_X(x)$ with state space $|S| < \infty$. This is accomplished by constructing a Markov chain in the space S that converges to the target distribution $P_X(x)$. If $q(x|y)$ is a transition kernel for a Markov chain, it can be proved the sufficient condition for the stationary distribution of the Markov chain to converge to $P_X(x)$ is that

$$\frac{q(x|y)}{q(y|x)} = \frac{P_X(x)}{P_X(y)}$$

This equation is called the equation of detailed balance. Suppose we sample from the Markov chain, and accept a sample with an *acceptance probability*. The equation of detailed balance is satisfied if the acceptance probability is chosen as follows:

$$A(x|y) = \min \left\{ 1, \frac{P_X(x)q(y|x)}{P_X(y)q(x|y)} \right\}$$

Markov Chain Monte Carlo approach constructs a Markov Chain with a proposal distribution $q(x|y)$ and an acceptance probability $A(x|y)$ such that the stationary distribution is the required target distribution $P_X(x)$. After reaching the target distribution, we can sample and estimate the required function/feature. Note that we are interested in the ratio of probabilities for P_X , not the absolute probabilities.

In the structure MCMC approach, a Markov chain is defined such that the states correspond to individual BN structures. Each step in the chain corresponds to a local transformation on the structure: adding, deleting, or reversing an edge. The proposal distribution is uniform over these local transformations, as defined below.

$$q(G|G') = \begin{cases} \frac{1}{N(G')} & \text{if } G \in N(G') \\ 0 & \text{otherwise} \end{cases}$$

where $N(G')$ is the neighborhood of G' , set of all DAGs that can be reached from the current graph state G' by deletion, addition or reversal of a single edge. If the acceptance probabilities are set to

$$A(G|G') = \min \left\{ 1, \frac{P(G|D)q(G'|G)}{P(G'|D)q(G|G')} \right\}$$

then the stationary distribution of the chain is the posterior $P(G|D)$ [Grzegorzczak et al., 2008]. If $f(G)$ is any feature, and the chain has converged to the posterior distribution $P(G|D)$, then

$$E_{P(G|D)}[f(G)] \approx \frac{1}{T} \sum_{t=1}^T f(G_t)$$

where T is the total samples sampled, and G_t is the t -th sampled graph.

5.4.2 Order MCMC

Order MCMC [Friedman and Koller, 2003] adds the following two constraints:

- Structures are consistent with some known order π i.e., the structures G s.t. if $X_i \in Pa(X_j)$ in G , then i precedes j in π .
- The number of parents any X_i can have is restricted to k

Let $\mathcal{G}_{k,\pi}$ be the set of Graphs with above two constraints. In Model Averaging, the problem is to find $P(f(G), D|\pi)$. By Bayes rule

$$P(f(G)|\pi, D) = \frac{P(f(G), D|\pi)}{P(D|\pi)}$$

In the above expression, the denominator is calculated as follows:

$$P(D|\pi) = \sum_{G \in \mathcal{G}_k} P(G|\pi)P(D|G)$$

The summation is still exponentially large. But, with the restriction to structure of a given order π , choice of parents for one node places no additional constraints on the choice for another. The possible sets for node X_i is $\mathcal{U}_{i,\pi} = \{U : U \text{ precedes } X_i \text{ in } \pi, |U| \leq k\}$. Therefore,

$$\begin{aligned} P(D|\pi) &= \sum_{G \in \mathcal{G}_{k,\pi}} \prod_i \text{LocalScore}(X_i, Pa_G(X_i), D) \\ &= \prod_i \sum_{U \in \mathcal{U}_{i,\pi}} \text{LocalScore}(X_i, U, D) \end{aligned}$$

The number of possible parents for each X_i is ${}^nC_k \leq n^k$. Hence total cost of computing $P(D|\pi)$ is at most n^{k+1} .

The numerator is sum over all structures that contain the feature and are consistent with the order.

$$P(f(G), D|\pi) = \sum_{G \in \mathcal{G}_{k,\pi}} f(G)P(G|\pi)P(D|G)$$

$P(D|G)$ has been derived earlier. The above term depends upon the feature. Friedman and Koller [2003] give expressions for Edge Feature, Markov Feature, and Parent Set Feature – all of which can be expressed in terms of LocalScore w.r.t. set $\mathcal{U}_{i,\pi}$.

Since it is computationally very expensive to consider all the structures consistent with all $n!$ possible orders, a Markov Chain Monte Carlo Algorithm is used. A uniform prior over order π is assumed. It is important to note that even though, π is uniform, $P(G)$ is not uniform: graphs that are consistent with more orders are more likely. For example, a Naive Bayes graph is consistent with $(n-1)!$ order, where as a chain structured graph is consistent with only one.

A Markov Chain is constructed with state space $\mathcal{O}$ consisting of all orders π ; if the construction guarantees that the chain has stationary distribution $P(\pi|D)$, any function $g(\pi)$ can be approximated as

$$E[g|D] \approx \frac{1}{T} \sum_{t=1}^T g(\pi_t)$$

Specifically, g here can be $P(f(G)|\pi, D)$ for some feature f . For the construction of the Markov chain, standard Metropolis sampling is used with a proposal probability $q(\pi'|\pi)$ and an acceptance probability

$$\min \left[1, \frac{P(\pi'|D)q(\pi|\pi')}{P(\pi|D)q(\pi'|\pi)} \right]$$

The operator considered for the construction is that two nodes are flipped in the order (leaving all other nodes unchanged). This operator can preserve much of computations in earlier iterations.

MODULE NETWORKS

Clustering is a common method used to partition a set of genes into groups such that genes belonging to the same group share the same set of characteristics. In the case of regulatory networks, the goal is to group genes with similar regulatory mechanism into cluster. In this section, we discuss two methods that group genes into clusters or modules based on the expression profiles. In the Agglomerative clustering method [Friedman, 2003], an algorithm to construct a partition of genes is developed using maximum likelihood and Bayesian methods. Segal et al. [2003a] use the framework of Bayesian networks to develop Module Network, where genes that are grouped into modules are regulated by the same set of genes, and hence have the same conditional probability distribution of expression profiles. Segal et al. [2003b] applied the method of module networks successfully to *Saccharomyces cerevisiae* expression data set. They tested all the modules for enrichment of gene annotations and regulatory binding site motifs.

6

6.1 PROBABILISTIC AGGLOMERATIVE CLUSTERING

The goal of any clustering method in gene network construction is to identify groups of genes such that genes with *similar* expression patterns are grouped together. Algorithms differ in the measures they use for similarity and the way output partitions are arranged in a hierarchy or as a set. Friedman [2003] describes a clustering procedure that is based on probabilistic considerations. Instead of similarity, a likelihood measure is used as the scoring method. Here, it is assumed user provides an input set of conditions, under which expression profiles are measured.

The input matrix D is viewed as a set $D = \{e_{g_i, c_k} : g_i \in S_G, c_k \in C\}$, where S_G is the set of genes and C is the set of conditions under which the expressions are measured. Suppose we have a partition $\mathcal{C} = \{C_1, \dots, C_p\}$ and $\mathcal{G} = \{S_{G_1}, \dots, S_{G_q}\}$, and we want to score the combined partition. Likelihood function is of the form

$$L(\mathcal{G}, \mathcal{C}, \theta : D) = P(D|\mathcal{G}, \mathcal{C}, \theta)$$

$$= \prod_i \prod_k \left[\prod_{g \in S_{G_i}} \prod_{c \in C_k} P(e_{g,c} | \theta_{i,k}) \right]$$

where $\theta_{i,k}$ are the parameters that describe the genes of S_{G_i} in conditions in C_k . With an assumption of the Gaussian distribution for each *cell*(i, k), the problem is then to estimate $\mu_{i,k}$ and variance $\sigma_{i,k}^2$. Maximum likelihood score is $\text{score}_{\text{ML}}(\mathcal{G}, \mathcal{C} : D) = \max_{\theta} L(\mathcal{G}, \mathcal{C}, \theta : D)$. By plugging in the maximum likelihood estimates of mean and variance, the maximum likelihood score can be computed. Since the maximum likelihood might be an overestimate, an alternate is the Bayesian approach. In the Bayesian approach, we average over all possible θ .

$$\text{score}_B(\mathcal{G}, \mathcal{C} : D) = \int P(D|\mathcal{G}, \mathcal{C}, \theta) P(\theta|\mathcal{G}, \mathcal{C}) d\theta$$

With the assumption of a normal-gamma prior, and prior independence, the Bayesian score of a cluster can be derived. Both ML and Bayesian scores have the form

$$\text{score}(\mathcal{C}, \mathcal{G} : D) = \prod_i \prod_k \text{LocalScore}(S_{G_i}, C_k : D)$$

where $\text{LocalScore}(G_i, C_k)$ is the local score of a particular cell. Each of the cell specific terms can be written as a function of vector of sufficient statistics. Let $s(x) = [1, x, x^2]^T$ be a vector valued function.

$$S_{i,k} = \sum_{g \in S_{G_i}} \sum_{c \in C_k} s(e_{g,c})$$

is defined as the sufficient statistics of cell (i, k) . Both maximum likelihood score and Bayesian score can be written as functions of statistics $S_{i,k}$.

Agglomerative Clustering problem here is that given an input partition of conditions $\mathcal{C} = \{C_1, \dots, C_p\}$, learn a partition of genes $\mathcal{G}$ that maximizes the score of $\mathcal{C}$ and $\mathcal{G}$. The algorithm starts with maximal partition of genes, where each gene has its own cluster. At the t -th stage of the algorithm, for each pair i and j of gene clusters, the change in score that results from merging S_{G_i} and S_{G_j} into a single cluster (denoted $\delta_{i,j}^{(t)}$) is calculated. Then the merger with the maximum $\delta_{i,j}^{(t)}$ is merged into a single cluster. The merging-step is repeated until we have a single cluster. It can be seen that each step in this process is quadratic in the size of $\mathcal{C}^{(t)}$. In practice it is only linear in size of $\mathcal{C}^{(t)}$, since for most pairs the $\delta_{i,j}$ doesn't change, and the only values that are to be computed involve the cluster created in the last step of the algorithms. Overall cost is $O(|S_G|^2 \mathcal{C})$. Stopping criteria can be modified to select a partition with maximum score, rather than a single final cluster.

The algorithm can be extended to cluster both genes and conditions at the same time. Start with some initial partition of conditions and then use agglomerative clustering to partition the genes, selecting the best scoring partition of genes. Then the partition of genes is fixed, and agglomeration is performed on conditions. Iterations are repeated until they converge. Although in theory, the iterations may not converge, it works well in practice.

6.2 LEARNING MODULE NETWORKS

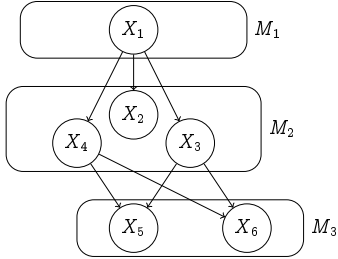


Figure 6.1: A module network with 6 variables and 3 modules. Each module has the same conditional probability distribution.

Module networks is a special case of Bayesian Network in which variables in the same module share the same set of parents and parameters. The goal of a Module network construction is to find set of variables with related behavior. Here, we discuss the learning algorithm developed by Segal et al. [2003a].

A module set $\mathcal{C}$ is a partition of variables into modules $M_1, \dots, M_k$, where all the modules within the same partition M_i share the same Conditional Probability distribution (CPD) and must have same domain values, denoted by $\text{Val}(M_j)$. A module network has the following two components:

1. A module network template $\mathcal{T} = (\mathcal{S}, \theta)$ for a module network $\mathcal{C}$ defines, for each module $M_j \in \mathcal{C}$: a set of parents $\text{Pa}(M_j) \subset \mathcal{X}$ and a conditional probability template (CPT) $P(M_j | \text{Pa}(M_j))$.
2. A module assignment function $\mathcal{A} : \mathcal{X} \rightarrow \{1, \dots, K\}$ such that $\mathcal{A}(X_i) = j$ only if $\text{Val}(X_i) = \text{Val}(M_j)$

A module network is defined as a triple $\mathcal{M} = (\mathcal{C}, \mathcal{T}, \mathcal{A})$. $\mathcal{M}$ defines a directed module graph $\mathcal{G}_{\mathcal{M}}$ with the nodes corresponding to the modules in $\mathcal{C}$ and the edge $M_j \rightarrow M_k$ if and only if there is a variable X s.t. $\mathcal{A}(X) = j$ and $X \in \text{Pa}(M_k)$.

The goal is to learn $\mathcal{A}$, $\mathcal{S}$ and θ from the data D using a *score-based* approach. The model score for a pair $(\mathcal{S}, \mathcal{A})$ given data D is defined in terms of the posterior probability of the pair, summing over all possible values of the parameters θ . By Chain rule,

$$P(\mathcal{S}, \mathcal{A} | D) \propto P(\mathcal{A}) P(\mathcal{S} | \mathcal{A}) P(D | \mathcal{S}, \mathcal{A})$$

where the term

$$P(D | \mathcal{S}, \mathcal{A}) = \int P(D | \mathcal{S}, \mathcal{A}, \theta) d\theta$$

The Bayesian score of a module network is defined as

$$\text{score}_B(\mathcal{S}, \mathcal{A} : D) = \log P(\mathcal{A}) + \log P(\mathcal{S}|\mathcal{A}) + \log P(D|\mathcal{S}, \mathcal{A})$$

To compute the score efficiently, the following assumptions are made: Parameter Independence, Parameter Modularity and Structure Modularity (discussed in the earlier section § 5.1 (p. 16)), Assignment Independence and Assignment Modularity. Assignment Independence states that $P(\theta|\mathcal{S}, \mathcal{A}) = P(\theta|\mathcal{S})$ and $P(\mathcal{S}|\mathcal{A}) = P(\mathcal{S})$. Assignment Modularity states that $P(\mathcal{A}) \propto \prod_j \alpha_j(\mathcal{A}_j)$, where $\mathcal{A}_j$ is the choice of variables assigned to module M_j and $\alpha_j : 2^{\mathcal{X}} \rightarrow \mathbf{R}$. The two assignment assumptions imply that the re-assignment of one variable from one module M_i to another module M_j doesn't change the preferences on assignments of variables in other modules. When the priors satisfy the assumptions, Bayesian score decomposes into module local scores:

$$\text{score}(\mathcal{S}, \mathcal{A} : D) = \sum_{j=1}^K \text{score}_{M_j}(Pa(M_j), \mathcal{A}(X^j) : D)$$

Score can be computed from the sufficient statistics, when we choose appropriate conjugate priors, and can be computed through a single pass of the data D .

The learning algorithm repeats the following two steps in each iteration

Structure Search Step learns the structure $\mathcal{S}$ assuming that $\mathcal{A}$ is fixed. A standard heuristic search over the space of dependency structures is used, similar to Bayesian Network structure learning. The search space is the state of legal parent structures of the modules, and the search operators take us from one state to another. In many cases, an obvious choice of local search operators involves steps of adding or removing a variable X_i from a parent set $Pa(M_j)$. Note that edge reversal is not a well-defined operator for module networks, as an edge from a variable to a module represents a one-to-many relation between the variable and all of the variables in the module. After applying an operator to $Pa(M_j)$, only the delta score for those operators that involve M_j are to be updated.

Module Assignment Search Step learns the assignment function fixing the structure $\mathcal{S}$ as $\mathcal{A} = \arg \max_{\mathcal{A}'} \text{score}(\mathcal{S}, \mathcal{A}' : D)$. Decomposition of the score among the variables, such that the optimal assignment $\mathcal{A}(X_i)$ is determined independently, doesn't work. For example, we can not simultaneously assign $\mathcal{A}(X_1) = j$ and $\mathcal{A}(X_2) = i$ when $X_1 \in Pa(M_i)$ and $X_2 \in Pa(M_j)$. The constraint that graph has to remain acyclic has to be maintained. Hence, we can only compute the delta score for moving a variable from one module to another, given a fixed assignment of the other variables to the two modules involved in the flip. Start with an initial assignment function A_0 , and in a "round-robin" fashion iterate over all of the variables one at a time, continue reassigning variables until no single reassignment can improve the score. When reassigning a variable X_i from one module M_j to another M_k , only the local score of these modules changes.

The initial module assignment to begin the iterative algorithm is chosen by an clustering algorithm similar to agglomerative clustering.

Segal et al. [2003a] use a regression tree the conditional probability model for the distribution $P(M_j|Pa(M_j))$. Regression tree T for a conditional distribution $P(X|U)$ is defined as a rooted binary tree. Each leaf l node corresponds to a univariate normal distribution over values of X . Each interior node is labeled with a test $U_i < u, u \in \mathbf{R}$ on some variable $U_i \in U$, and its two children correspond to the outcome of the test. Learning using regression trees adds another component to $\mathcal{S}$ that represents the trees $T_1, \dots, T_K$ associated with K modules. During structure search, in addition to choosing parents for each module, an associated tree structure has also needs to be

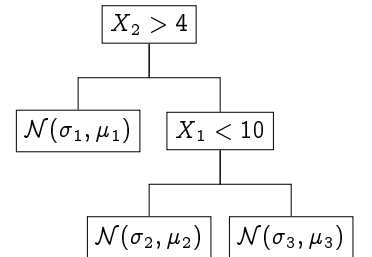


Figure 6.2: Regression Tree example for a variable with two parents X_1 and X_2 .

chosen. The search operator used here is tree splits, which replaces an existing leaf node with an internal node, that test some variable $U_i \in U$. Two new branches point to two leaves each with associated Gaussian.

PERFORMANCE

In this chapter, we discuss the performance of methods we have discussed in the earlier chapters. In developing a Gene network model, we are interested the number of true positives (edges correctly predicted), the number of true negatives (edges correctly avoided), the number of false positives (edges incorrectly predicted), and the number false negatives (incorrectly avoided edges) – denoted TP, TN, FP, and FN respectively. The standard measures used to evaluate the performance of a method in developing Gene Networks are



- 1 Specificity – fraction of missing edges correctly classified $\frac{TN}{TN+FP}$.
- 2 Sensitivity – fraction of correct edges predicted $\frac{TP}{TP+FN}$.
- 3 Precision – fraction of predicted edges that are correct $\frac{TP}{TP+FP}$.

7.1 RELEVANCE NETWORKS

It is common practice to validate network inference algorithms using synthetic data sets generated by tools such as SynTReN, COPASI, RANGE and NEMO. They provide a functionality to create benchmark data sets with required sizes. Zola et al. [2010] generated two synthetic regulatory networks using SynTReN, each consisting of 250 genes, but differing in the number of expression profiles. Zola et al. [2010] outperform Basso et al. [2005] in execution time while preserving high precision and sensitivity. The parallel algorithm of Zola et al. [2010] maintains linear scalability up to 1,024 CPUs. In each case, the total execution time is dominated by pairwise MI computations (first phase), with the second and third phases of the algorithm accounting for less than 5 seconds in the worst case. Zola et al. [2010] also develop an algorithm for cell cluster, with which they constructed a 15,222 gene network covering the complete genome of the plant *Arabidopsis thaliana*.

7.2 GAUSSIAN GRAPHICAL MODELS

Schäfer and Strimmer [2005] used a simulation setup, which can generate data controlling various parameters of interest. Performance measure of various $\hat{\Pi}$ estimators of GGM are evaluated using count of TP, FP, TN and FN edges and Sensitivity and specificity. These evaluations are used only to select the best estimator of $\hat{\Pi}$. There is no comparison done against other methods. Schäfer and Strimmer [2005] analyze gene expression data from a breast cancer study, and validate the parts of the learned network against the domain knowledge.

Ma et al. [2007] used GGM coupled with iterative random samplings of genes was expanded into a network that covered the Arabidopsis genome (22,266 genes). Partial correlation between gene pairs are calculated among 2000 input genes at a time. GGM displayed known coregulation pathways as sub networks, and reconciled individual subnetworks in a topology joined at the whole-genome level. It also added novel components to known edges.

7.3 BAYESIAN NETWORKS

Ott et al. [2004] used the *Saccharomyces cerevisiae* stress response data, and found the results to be biologically possible. Because of the time complexity, they also declare that their implementation will not allow to estimate networks of more than about 40 genes. Nikolova et al. [2009] used the SynTReN package to generate synthetic data sets for evaluation. They evaluated how number of observations influence scalability, fixing the number of genes. They found that the execution time increases linearly with the number of observations, and at the same time the implementation maintains linear scalability up to 1,024 CPUs. They also studied the performance of the parallel algorithm with respect to the number of input variables. Using 1,024 processors, the implementation was able to infer the Bayesian network for 30 genes in about 1.5 hours. Note that in exact learning algorithm developed in both Ott et al. [2004] and Nikolova et al. [2009], the memory required grows exponentially with n

Friedman and Koller [2003] compared Order MCMC with a full Bayesian model averaging, in small domains where exhaustive enumeration of Bayesian network structures is practical. They evaluated that results of the sampling process are close to the results of true Bayesian model averaging. They found a very close correlation between the estimate of Order MCMC and the exact computation for both Markov features and Edge feature, even with small sample sizes. Then, they compared Order MCMC with the Structure MCMC approach. Here, they compare of the mixing rate and the estimates of the two MCMC samplers. They found Order MCMC runs mix much faster compared to Structure MCMC. They believe that the difference in mixing rate is due to the difference in landscape of the space of orders and structures. When comparing the feature estimates of the two MCMC methods, they are not too far apart, although the posterior estimate of the structure-MCMC is usually larger. Also, in constructing the structure, order-MCMC does as well or better than the other approaches. The features it misses are possibly due to weak interactions.

7.4 MODULE NETWORKS

Segal et al. [2003b] constructed a module networks of 2,355 genes yeast stress data set using an input list of 466 candidate regulators. Module Network algorithm created a network of 50 modules. They evaluated all 50 modules to test whether the proteins encoded by genes in the same module had related functions. They are able to identify many modules related to a particular function, and validate many genes against information available in gene annotations database. They also experimentally validate three hypotheses suggested by this method, involving uncharacterized putative regulators.

Segal et al. [2003b] note the cases where Module Networks may fail.

- The method may fail to identify certain regulatory relations and may occasionally predict regulatory relationships that do not hold. Probable causes are insufficient regulator expression and post-transcriptional changes.
- If several regulators participate in the same regulatory event, the method typically identifies only one representative of this group.
- Some regulatory relations may be specific to a regulator and its target and cannot be generalized to an entire module.

7.5 COMPARISON STUDY

Grzegorzczuk et al. [2008] performed a comparison study of three of the methods discussed earlier: Relevance networks, Gaussian Graphical models, and Bayesian networks with both Structure MCMC and Order MCMC. For relevance networks, they used Pearson correlation coefficient as the relevance measure. They used RAF signaling pathway for comparison, a well-known regulatory network with 11 molecules

and 1200 observational measurements. The RAF pathway has been extensively studied. They used both real laboratory data and synthetic simulated data based on the rationale that the true structure of the regulatory network is known for the simulated data and hence, in principle the prediction results can be faithfully evaluated.

Results suggest that when the posterior probabilities of true and false edges are computed, GGMs show a clearer separation of true and false edges than Relevance networks. But this does not translate to an improved performance of GGMs over Relevance Networks in terms of TP scores. Even though the separation of scores is poorer for Relevance Networks, it has not affected the ranking of edges. It suggests that inference with Relevance Networks is less stable than Gaussian Graphical Models.

In comparing GGMs with Bayesian Networks, the results showed no significant difference between Bayesian networks and GGMs for observational data. For interventional data, Bayesian networks outperform GGMs and Relevance networks, when edge directions are taken into account. Grzegorzczuk et al. [2008] conclude that higher computational cost of Bayesian Networks over GGMs or Relevance Networks are not justified for passive observations, but that active interventions such as gene knockouts are required to exploit the full potential of Bayesian networks.

CONCLUSION

In this report, the most commonly used methods for constructing gene regulatory networks are discussed. There is no clear “winner” among the methods, and there are no exhaustive comparative studies done on the relative pros and cons. The difficulty in performing comparative studies is that knowledge about many interactions of biological processes are incomplete. Also, different methods give different interpretation of networks, and the comparing among them is difficult without using a neutral measure. For example, Relevance networks and GGMs are undirected graphical models – providing the interpretation of association or correlation, while Bayesian networks are directed – providing the interpretation of dependence. So, just counting true positives or true negatives may not be a fair comparison.

Of the methods we have seen, only GGMs and Relevance networks are used for genome-wide construction of gene regulatory network. While the parallel algorithm implementation of MI Relevance networks [Zola et al., 2010] is able to handle the whole genome at a time, GGMs had to iterate through all the genes taking 2000 genes at a time Ma et al. [2007]. Genome-wide Gene Regulatory network probably needs a lot of work on parallel algorithm development.

Following are some of the questions, which can be explored further.

- How to develop priors that can exploit scale-free nature of the gene regulatory networks? Is it possible to use the power-law distribution as a prior for Bayesian networks?
- How to use Bayesian networks for genome-wide gene network construction? How to make Bayesian network algorithms parallel?
- How can we exploit modular nature of networks in developing parallel algorithms?
- Is it possible to use node replacement grammars as priors?

The approaches given in this report are not the exhaustive. Dynamic Bayesian Networks, Probabilistic Boolean Networks, Linear Regression with LASSO and Robust Regression are some of the other approaches used. Also, Gene networks are constructed from data other than expression profiles such as sequence mapping, and promoter sequence. There are also hybrid methods that combine promoter sequence and expression profiles. This report describes only the most common methods used.



BIBLIOGRAPHY

- Katia Basso, Adam A Margolin, Gustavo Stolovitzky, Ulf Klein, Riccardo Dalla-favera, and Andrea Califano. Reverse engineering of regulatory networks in human B cells. *Nature Genetics*, 37(4), 2005. 9, 10, 27
- A J Butte and I S Kohane. Mutual Information Relevance Networks:Functional Genomic Clustering Using Pairwise Entropy Measurements. *Pacific Symposium on Biocomputing. Pacific Symposium on Biocomputing*, 426:418–29, January 2000. ISSN 1793-5091. 8
- David Maxwell Chickering. Large-Sample Learning of Bayesian Networks is NP-Hard. *Journal of Machine Learning Research*, 5:1287–1330, 2004. 15
- Carsten O Daub, Ralf Steuer, Joachim Selbig, and Sebastian Kloska. Estimating mutual information using B-spline functions—an improved similarity measure for analysing gene expression data. *BMC bioinformatics*, 5:118, August 2004. ISSN 1471-2105. 8
- Nir Friedman. PCluster : Probabilistic Agglomerative Clustering of Gene Expression Profiles. pages 1–6, 2003. 23
- Nir Friedman and Daphne Koller. Being Bayesian About Network Structure . A Bayesian Approach to Structure Discovery. *Machine Learning*, pages 95–125, 2003. 20, 21, 28
- Dan Geiger and David Heckerman. Learning Gaussian Networks. In *UAI’94*, pages 235–243, 1994. 17
- Marco Grzegorzcyk, Dirk Husmeier, and Adriano V. Werhli. *Reverse Engineering Gene Regulatory Networks with Various Machine Learning Methods*, pages 100–141. 2008. 1, 20, 28, 29
- Alexander J Hartemink. Reverse engineering gene regulatory networks. *Nature Biotechnology*, 23(5):554–555, 2005. 9
- David Heckerman. *A tutorial on learning with Bayesian networks*, pages 301–354. MIT Press, Cambridge, MA, USA, 1999. ISBN 0-262-60032-3. 16
- David Heckerman and David M Chickering. Learning Bayesian Networks : The Combination of Knowledge and Statistical Data Metrics for Belief Networks. 1993. 16
- Daphne Koller and Nir Friedman. *Probabilistic Graphical Models:Principles and Techniques*. The MIT Press, 2009. 8
- Shisong Ma, Qingqiu Gong, and Hans J Bohnert. An Arabidopsis gene network based on the graphical Gaussian model. *Genome Research*, 17(11):1614–25, November 2007. ISSN 1088-9051. 27, 31
- Olga Nikolova, Jaroslaw Zola, and Srinivas Aluru. A Parallel Algorithm for Exact Bayesian Network Inference. *IEEE*, pages 342–349, 2009. 18, 28

- S. Ott, S. Imoto, and S. Miyano. Finding Optimal Models for Small Gene Networks. In *Pacific Symposium on Biocomputing*, volume 567, pages 557–567, 2004. 17, 18, 27, 28
- Andrzej Polanski and Marek Kimmel. *Bioinformatics*. Springer Publishing, 2007. 4
- Charlotte W. Pratt and Kathleen Cornely. *Essential Biochemistry*. John Wiley & Sons, 2004. 4
- Juliane Schäfer and Korbinian Strimmer. An empirical Bayes approach to inferring large-scale gene association networks. *Bioinformatics (Oxford, England)*, 21(6): 754–64, March 2005. ISSN 1367-4803. 11, 12, 13, 27
- Eran Segal, Dana Pe’er, Daphne Koller, Nir Friedman, and Aviv Regev. Learning Module Networks. page 10, December 2003a. ISSN 0014-5793. 23, 24, 25
- Eran Segal, Michael Shapira, Aviv Regev, Dana Pe’er, David Botstein, Daphne Koller, and Nir Friedman. Module networks: identifying regulatory modules and their condition-specific regulators from gene expression data. *Nature genetics*, 34(2): 166–76, June 2003b. ISSN 1061-4036. 23, 28
- Xiaowei Zhu, Mark Gerstein, and Michael Snyder. Getting connected : analysis and principles of biological networks. *Genes & Development*, pages 1010–1024, 2007. 5
- Jaroslav Zola, Maneesha Aluru, Abhinav Sarje, Student Member, and Srinivas Aluru. Parallel Information-Theory-Based Construction of Genome-Wide Gene Regulatory Networks. *IEEE Transactions on Parallel and Distributed Systems*, 21(12): 1721–1733, 2010. 7, 9, 27, 31